

INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo (ESCOM / UPIIZ)

Ingeniería en Sistemas Computacionales

TEMA

**Número maravilloso
(conjetura de Collatz)**

Asignatura: Desarrollo de Aplicaciones Móviles Nativas

Tarea: Tarea 5: Número maravilloso

Alumno: Angel Frausto Robles

Grupo: 7CM4

Fecha: 12 de septiembre de 2026

INTRODUCCIÓN

Esta tarea pide construir una aplicación Android que reciba un número natural y determine si es un "número maravilloso", es decir, si al aplicarle repetidamente la regla de la conjetura de Collatz (dividir entre 2 si es par, multiplicar por 3 y sumar 1 si es impar) la secuencia siempre termina en 1.

Objetivos:

- Implementar la regla de Collatz con una estructura de control simple en Java.
- Mostrar al usuario la secuencia completa de pasos hasta llegar a 1, no solo el resultado final.
- Probar la aplicación con números de distinto tamaño para observar cómo varía la cantidad de pasos.

DESARROLLO

1. Entorno de trabajo

El proyecto se hizo en Java con el SDK clásico de Android: `MainActivity` extiende directamente `android.app.Activity`, sin usar `AppCompatActivity` ni otras clases de AndroidX. La interfaz usa vistas nativas del framework (`EditText`, `Button`, `TextView`), sin `ConstraintLayout`.

2. Interfaz (activity_main.xml)

Un `ScrollView` con un `LinearLayout` vertical adentro contiene el título, el campo para escribir el número, el botón "Verificar" y el `TextView` donde se muestra la secuencia completa. El `ScrollView` es necesario porque números como 27 generan una secuencia de más de cien pasos.

```
<ScrollView ...>
  <LinearLayout android:orientation="vertical" ...>
    <TextView android:text="@string/xtitulo" />
    <EditText android:id="@+id/xnumero" android:inputType="number"
      android:hint="@string/xdato" />
    <Button android:id="@+id/xboton" android:text="@string/xaceptar" />
    <TextView android:id="@+id/xresultado" android:fontFamily="monospace" />
  </LinearLayout>
</ScrollView>
```

3. Lógica (MainActivity.java)

Al presionar el botón, se valida que el campo no esté vacío y que el texto se pueda convertir a número. Si el número es válido, se construye la secuencia de Collatz en un ciclo `while` hasta llegar a 1, contando los pasos y guardando cada valor en un `StringBuilder` para mostrarlo completo al final:

```
StringBuilder secuencia = new StringBuilder();
secuencia.append(numero);
long n = numero;
int pasos = 0;
while (n != 1 && pasos < 1000) {
    if (n % 2 == 0) {
        n = n / 2;
    } else {
        n = 3 * n + 1;
    }
    secuencia.append(" -> ").append(n);
    pasos++;
}
boolean esMaravilloso = (n == 1);
```

El límite de 1000 pasos es solo una protección: la conjetura de Collatz nunca se ha demostrado para todos los números naturales, así que en teoría no hay garantía de que algún número extremo tarde más de eso en llegar a 1, aunque para los números que caben en un `long` nunca se ha encontrado uno así.

4. Pruebas en el emulador Android

Se probaron tres números para cubrir un caso corto, uno largo y el número que da nombre a esta tarea:

Número	Resultado	Pasos
--------	-----------	-------

6	Maravilloso	8
27	Maravilloso	111

Numero maravilloso (conjetura de Collatz)

Ingresar un numero natural

VERIFICAR

Figura 1. Pantalla inicial de la aplicación.

Numero maravilloso (conjetura de Collatz)

6

VERIFICAR

Numero analizado: 6

6 SI es un numero maravilloso.

Secuencia:
6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1

Pasos: 8

Figura 2. Número 6: secuencia corta, 8 pasos.

Numero maravilloso (conjetura de Collatz)

27

VERIFICAR

Numero analizado: 27

27 SI es un numero maravilloso.

Secuencia:
27 -> 82 -> 41 -> 124 -> 62 -> 31 -> 94 -> 47
-> 142 -> 71 -> 214 -> 107 -> 322 -> 161 ->
484 -> 242 -> 121 -> 364 -> 182 -> 91 -> 274
-> 137 -> 412 -> 206 -> 103 -> 310 -> 155 ->
466 -> 233 -> 700 -> 350 -> 175 -> 526 -> 263
-> 790 -> 395 -> 1186 -> 593 -> 1780 -> 890
-> 445 -> 1336 -> 668 -> 334 -> 167 -> 502 ->
251 -> 754 -> 377 -> 1132 -> 566 -> 283 ->
850 -> 425 -> 1276 -> 638 -> 319 -> 958 ->
479 -> 1438 -> 719 -> 2158 -> 1079 -> 3238 ->
1619 -> 4858 -> 2429 -> 7288 -> 3644 -> 1822
-> 911 -> 2734 -> 1367 -> 4102 -> 2051 ->
6154 -> 3077 -> 9232 -> 4616 -> 2308 -> 1154
-> 577 -> 1732 -> 866 -> 433 -> 1300 -> 650
-> 325 -> 976 -> 488 -> 244 -> 122 -> 61 ->
184 -> 92 -> 46 -> 23 -> 70 -> 35 -> 106 ->
53 -> 160 -> 80 -> 40 -> 20 -> 10 -> 5 -> 16
-> 8 -> 4 -> 2 -> 1

Pasos: 111

Figura 3. Número 27: 111 pasos, uno de los ejemplos clásicos de la conjetura.

CONCLUSIONES

Lo que más me llamó la atención de esta tarea es la diferencia de tamaño entre la lógica y el resultado. La regla en sí son dos líneas: dividir entre 2 o multiplicar por 3 y sumar 1. Pero el número 27, que no tiene nada de especial a simple vista, tarda 111 pasos en llegar a 1, subiendo hasta más de 9000 en el camino antes de bajar. Ver la secuencia completa en pantalla, en lugar de solo un "sí" o un "no", ayuda a entender por qué a este problema se le sigue llamando conjetura y no teorema: nadie ha probado que funcione para todos los números, solo que no se ha encontrado ningún contraejemplo.

BIBLIOGRAFÍA

- Cifuentes A., Alejandro. *DAMN Kotlin Practica4* (documento de la materia donde se planteó por primera vez el número maravilloso).
- Android Developers. *Activity, Views and Layouts*. <https://developer.android.com/guide>
- Wikipedia. *Collatz conjecture*. https://en.wikipedia.org/wiki/Collatz_conjecture